

Authentication / Authorization

Authentication = Who are you?

Authorization = What are you allowed to access?

role = "user"

role = "admin"

Frontend - Read Login Fields

```
const email =
  document.getElementById("email").value.trim();
const password =
  document.getElementById("password").value;
```

Frontend - Create Login Object

```
const credentials = {
  email: email,
  password: password
};
```

Frontend - POST Login Request

```
const response = await fetch("/api/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(credentials)
});
const data = await response.json();
```

Frontend - Check Response

```
if (!response.ok) {
  // display data.error
}
```

Frontend - Redirect by Role

```
if (data.role === "admin") {
  window.location.href = "/admin";
} else {
  window.location.href = "/user";
}
```

MongoDB Connection

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["database_name"]
users_collection = db["users"]
```

Find One User

```
user = users_collection.find_one({
  "email": email
})
```

Read JSON in Flask

```
data = request.get_json() or {}
email = data.get("email")
password = data.get("password")
```

Password Hash Check

```
from werkzeug.security import check_password_hash
check_password_hash(
  user["password_hash"],
  password
)
```

Create Password Hash

```
from werkzeug.security import generate_password_hash
password_hash =
  generate_password_hash(password)
```

Flask Session

```
session["user_id"] = str(user["_id"])
session["name"] = user["name"]
session["email"] = user["email"]
session["role"] = user["role"]
```

Check Login Session

```
if "user_id" not in session:
  # user is not logged in
```

Check Admin Role

```
if session.get("role") != "admin":
  return "Forbidden", 403
```

Protected User Route Pattern

```
@app.get("/user")
def user_page():
  if "user_id" not in session:
    return redirect("/")
  # return user page
```

Protected Admin Route Pattern

```
@app.get("/admin")
def admin_page():
    if "user_id" not in session:
        return redirect("/")
    if session.get("role") != "admin":
        return "Forbidden", 403
    # return admin page
```

Current User API Pattern

```
@app.get("/api/me")
def current_user():
    if "user_id" not in session:
        return jsonify({
            "error": "Not authenticated"
        }), 401
    # return session user information
```

Logout

```
session.clear()
```

Useful Status Codes

```
200 OK
400 Bad Request
401 Not Authenticated
403 Forbidden
404 Not Found
```

Important Security Rules

Do not store plaintext passwords.
Do not put database passwords in JavaScript.
Do not trust only a frontend role redirect.
Protect admin routes on the backend.
Use the session to check authenticated access.

MongoDB Verification

```
use coffee_auth
show collections
db.users.find()
db.users.find(
    {},
    { password_hash: 0 }
)
```

Login Flow

```
Login Form
-> POST /api/login
-> Flask
-> MongoDB find_one()
-> check_password_hash()
-> session
-> role check
-> user page / admin page
```